LAPORAN PRAKTIKUM PERTEMUAN 5



Dosen Pengampu: Juni Ismail, S.Kom., M.Kom.

Disusun Oleh :

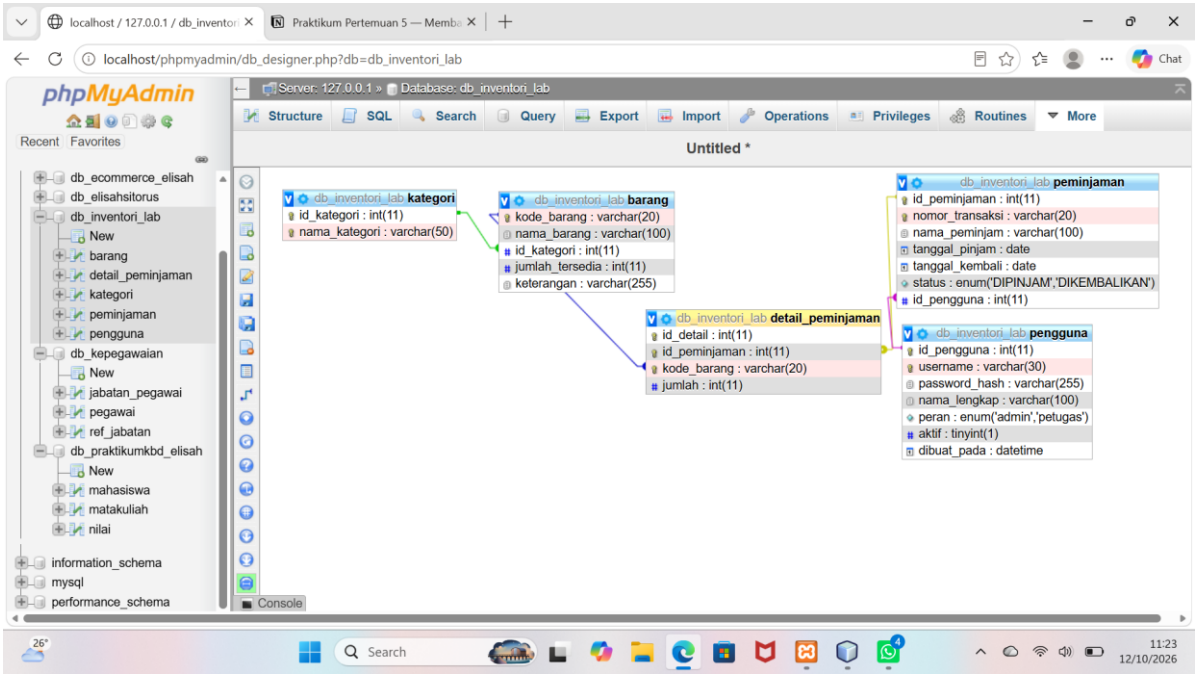
Nama : Arifa Zandy Nasution
Nim : 25110101
Prodi : Teknik Komputer

A. Tujuan dan Gambaran Umum

Praktikum ini bertujuan membangun dan menguji database inventori laboratorium bernama db_inventori_lab menggunakan MySQL yang dikelola lewat phpMyAdmin. Laporan ini memuat bukti tangkapan layar (screenshot) beserta penjelasan setiap perintah SQL, mulai dari rancangan relasi tabel, perintah menampilkan data (SELECT, WHERE, LIKE, JOIN, GROUP BY), perintah mengubah data (INSERT, UPDATE, DELETE), sampai pengujian aturan integritas data (primary key, foreign key, NOT NULL, dan CHECK constraint) yang sengaja dibuat gagal.

1. Desain Database dan Relasi Antartabel (Designer)

Gambar ini menampilkan fitur Designer pada phpMyAdmin untuk database db_inventori_lab. Designer memperlihatkan seluruh tabel beserta kolom, tipe data, dan garis relasi (foreign key) antartabel.



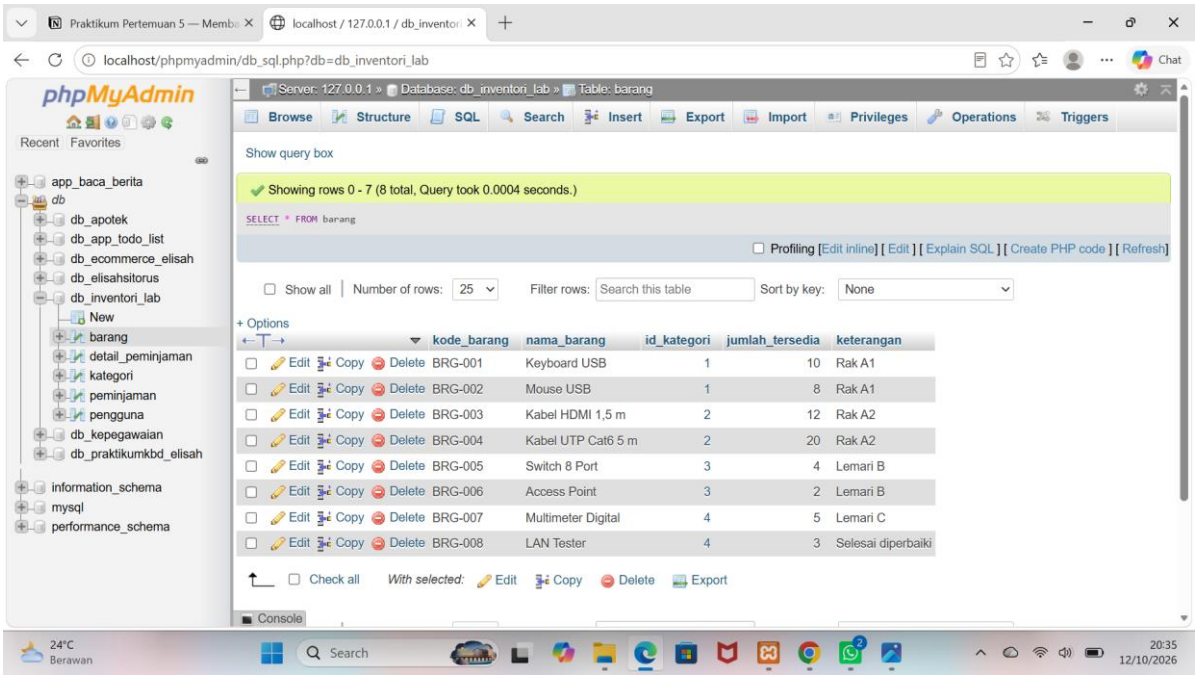
Hasil: Rancangan ini sudah ternormalisasi: data kategori, barang, petugas, dan transaksi dipisah ke tabel masing-masing dan dihubungkan dengan primary key serta foreign key agar data konsisten.

2. Menampilkan Seluruh Data Barang

Query ini menampilkan semua kolom dan semua baris dari tabel barang. Tanda * berarti "semua kolom".

Perintah SQL:

```
SELECT * FROM barang;
```



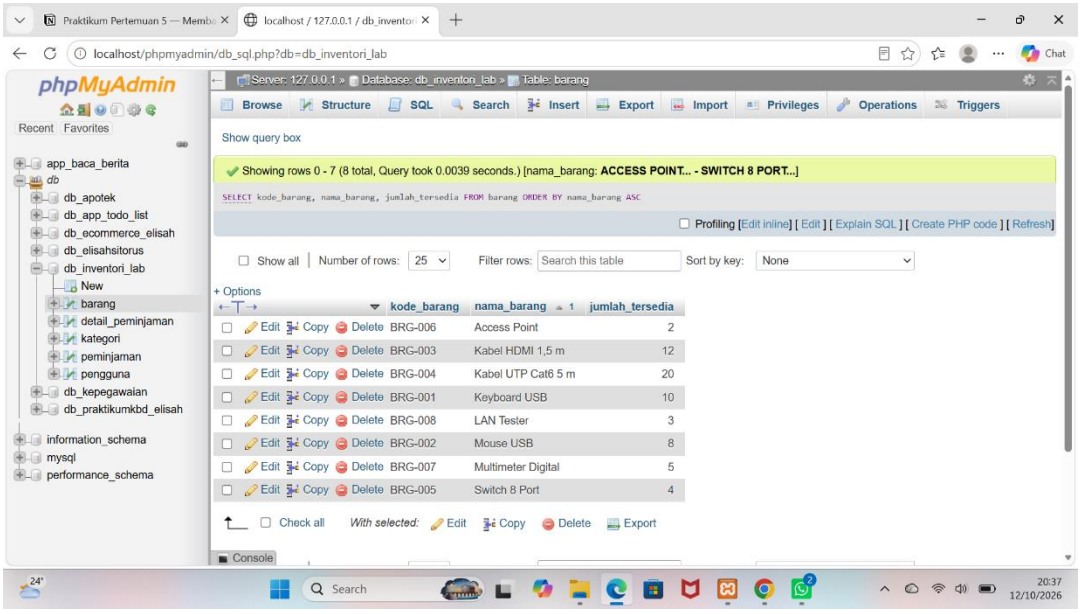
Hasil: Query SELECT * berguna untuk melihat isi tabel secara utuh dan memastikan data awal sudah masuk dengan benar.

3. Memilih Kolom Tertentu dan Mengurutkan Data

Query ini hanya mengambil tiga kolom (kode_barang, nama_barang, jumlah_tersedia) lalu mengurutkan hasilnya berdasarkan nama barang.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
ORDER BY nama_barang ASC;
```



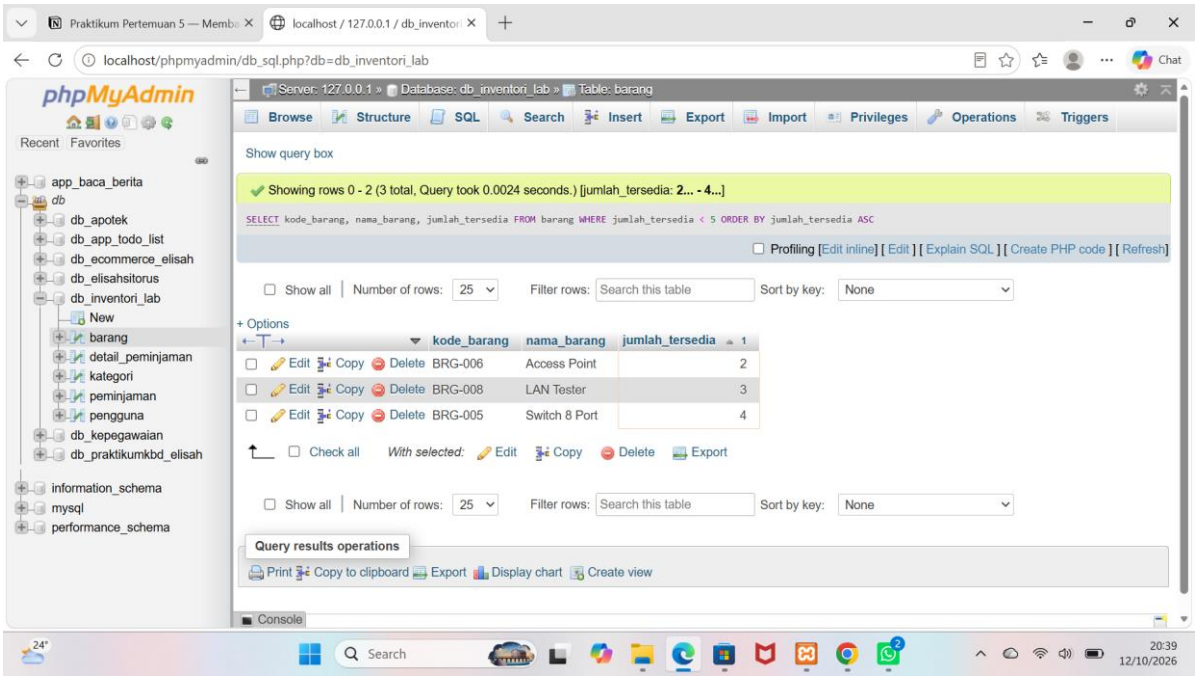
Hasil: Hasil menampilkan 8 baris yang tersusun alfabetis berdasarkan nama barang.

4. Menyaring Data dengan WHERE (Stok Menipis)

Query ini menampilkan barang yang stoknya menipis, yaitu jumlah tersedia kurang dari 5, diurutkan dari stok paling sedikit.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
WHERE jumlah_tersedia < 5
ORDER BY jumlah_tersedia ASC;
```



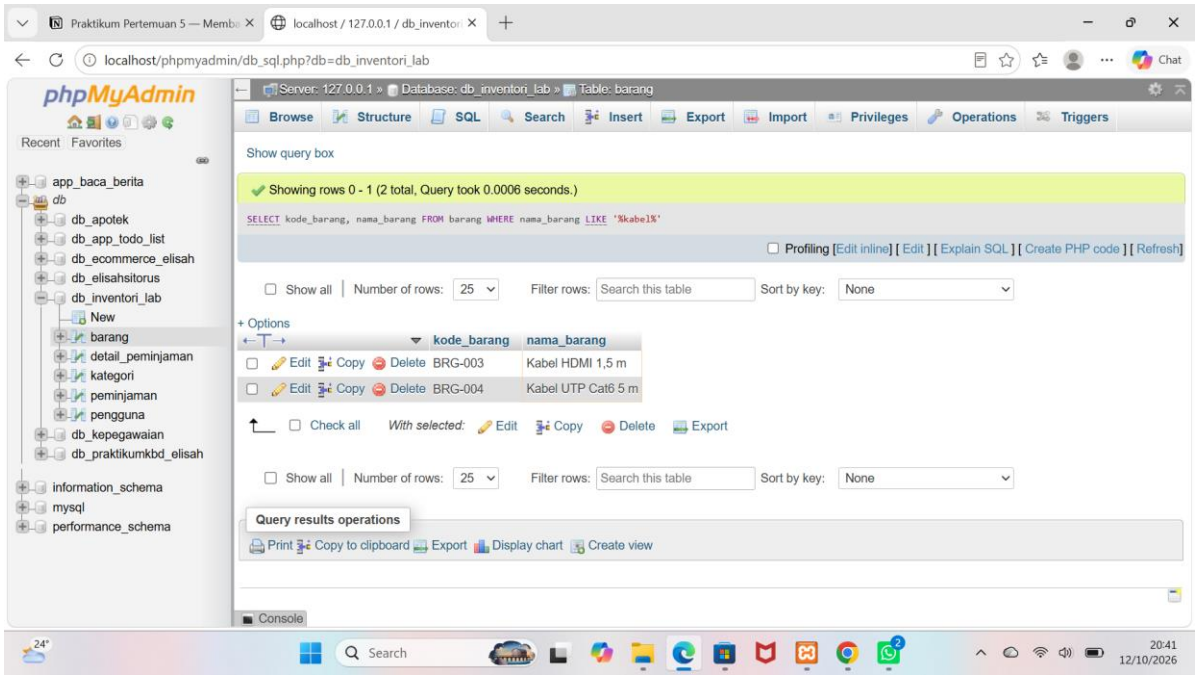
Hasil: Hasilnya 3 baris: Access Point (BRG-006) stok 2, LAN Tester (BRG-008) stok 3, dan Switch 8 Port (BRG-005) stok 4. Ketiganya perlu diprioritaskan untuk pengadaan atau perbaikan.

5. Pencarian Teks dengan LIKE

Query ini mencari barang yang nama_barang-nya mengandung kata "kabel" di posisi mana pun.

Perintah SQL:

```
SELECT kode_barang, nama_barang
FROM barang
WHERE nama_barang LIKE '%kabel%';
```



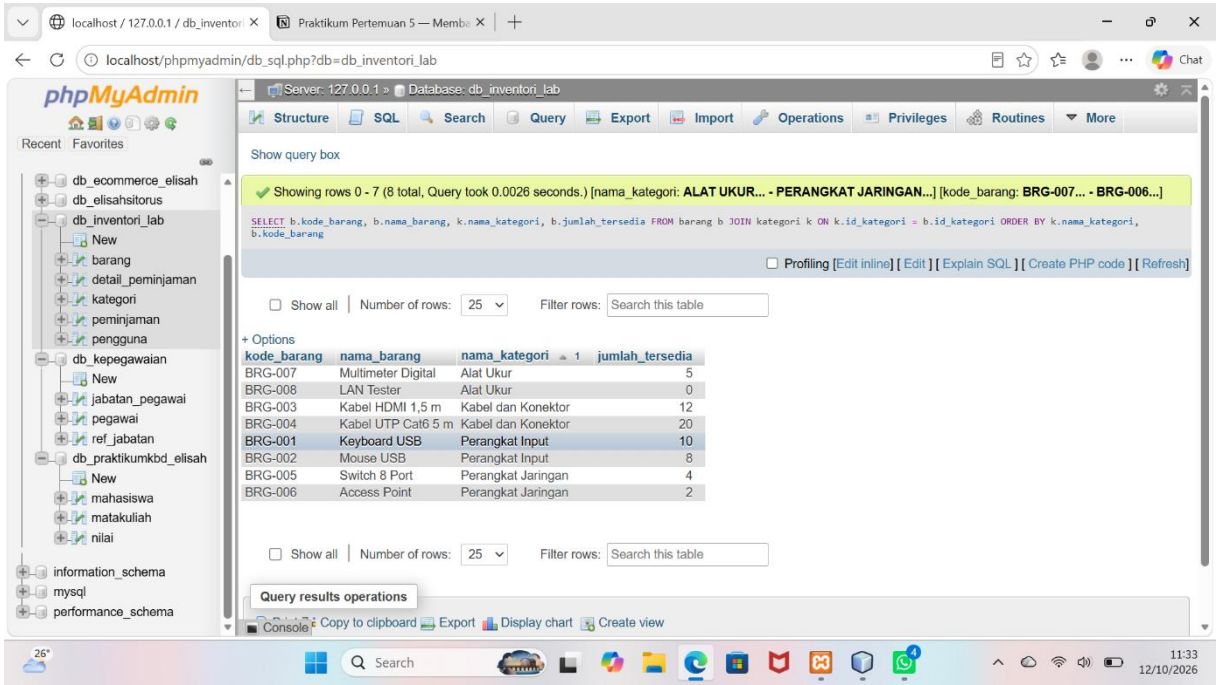
Hasil: Hasilnya 2 baris: Kabel HDMI 1,5 m (BRG-003) dan Kabel UTP Cat6 5 m (BRG-004).

6. Menggabungkan Dua Tabel dengan JOIN (barang dan kategori)

Tabel barang hanya menyimpan id_kategori (angka). Dengan JOIN, id tersebut dicocokkan dengan tabel kategori sehingga nama kategori dapat ditampilkan.

Perintah SQL:

```
SELECT b.kode_barang, b.nama_barang, k.nama_kategori, b.jumlah_tersedia
FROM barang b
JOIN kategori k ON k.id_kategori = b.id_kategori
ORDER BY k.nama_kategori, b.kode_barang;
```



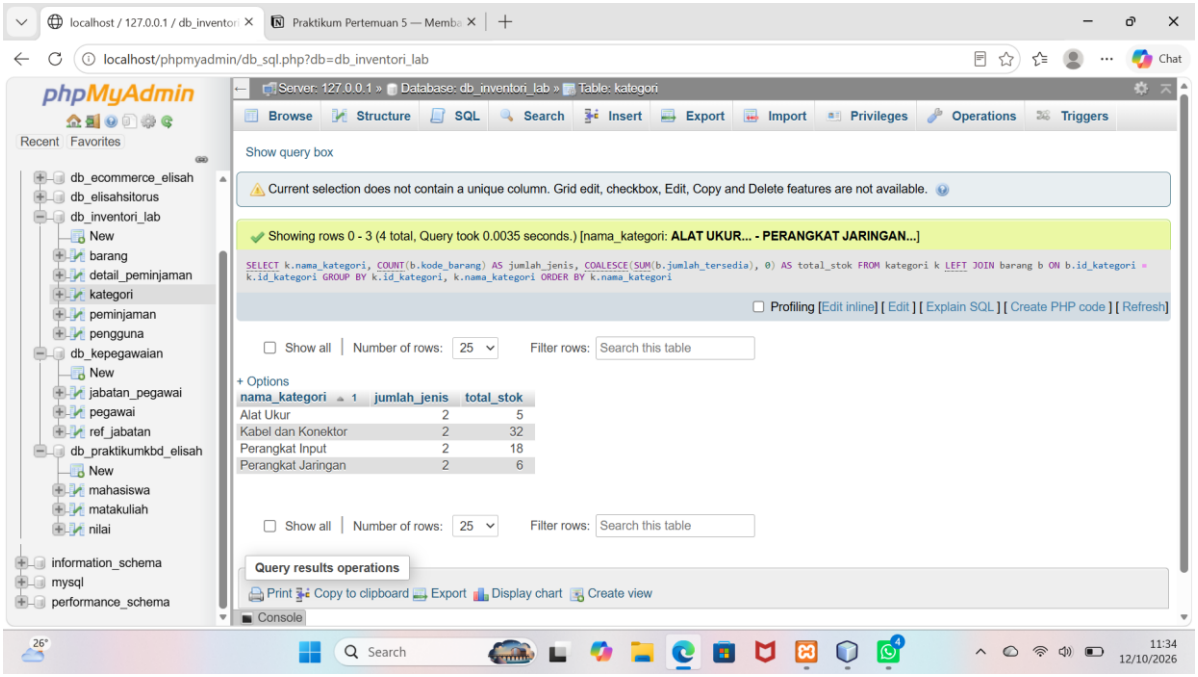
Hasil: Hasilnya 8 baris yang sudah memuat nama kategori sehingga lebih mudah dibaca daripada hanya menampilkan id_kategori.

7. Rekap per Kategori dengan LEFT JOIN, COUNT, SUM, dan GROUP BY

Query ini membuat rekap: untuk setiap kategori, berapa jenis barangnya dan berapa total stoknya.

Perintah SQL:

```
SELECT k.nama_kategori,
       COUNT(b.kode_barang) AS jumlah_jenis,
       COALESCE(SUM(b.jumlah_tersedia), 0) AS total_stok
FROM kategori k
LEFT JOIN barang b ON b.id_kategori = k.id_kategori
GROUP BY k.id_kategori, k.nama_kategori
ORDER BY k.nama_kategori;
```



Hasil: Hasilnya 4 kategori: Alat Ukur (2 jenis, stok 5), Kabel dan Konektor (2 jenis, stok 32), Perangkat Input (2 jenis, stok 18), dan Perangkat Jaringan (2 jenis, stok 6).

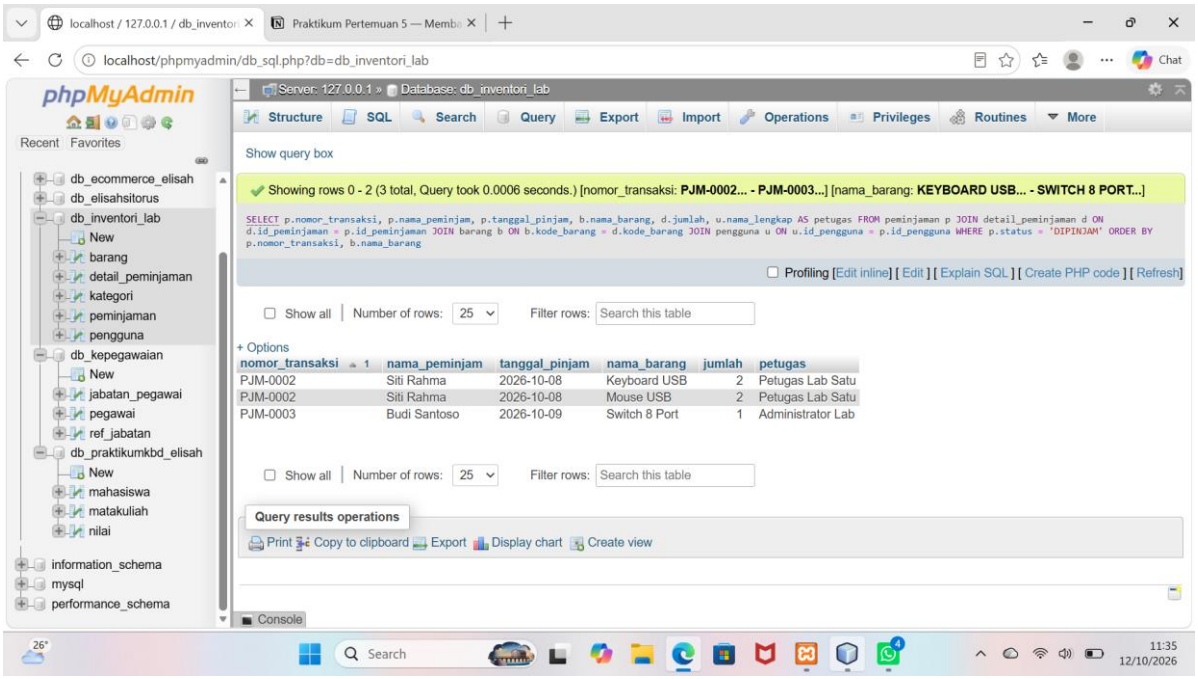
8. Laporan Peminjaman Aktif (JOIN Empat Tabel)

Query ini menggabungkan empat tabel (peminjaman, detail_peminjaman, barang, dan pengguna) untuk membuat laporan peminjaman yang masih berjalan.

Perintah SQL:

```
SELECT p.nomor_transaksi, p.nama_peminjam, p.tanggal_pinjam,
       b.nama_barang, d.jumlah, u.nama_lengkap AS petugas
FROM peminjaman p
JOIN detail_peminjaman d ON d.id_peminjaman = p.id_peminjaman
JOIN barang b ON b.kode_barang = d.kode_barang
JOIN pengguna u ON u.id_pengguna = p.id_pengguna
WHERE p.status = 'DIPINJAM'
```

```
ORDER BY p.nomor_transaksi, b.nama_barang;
```



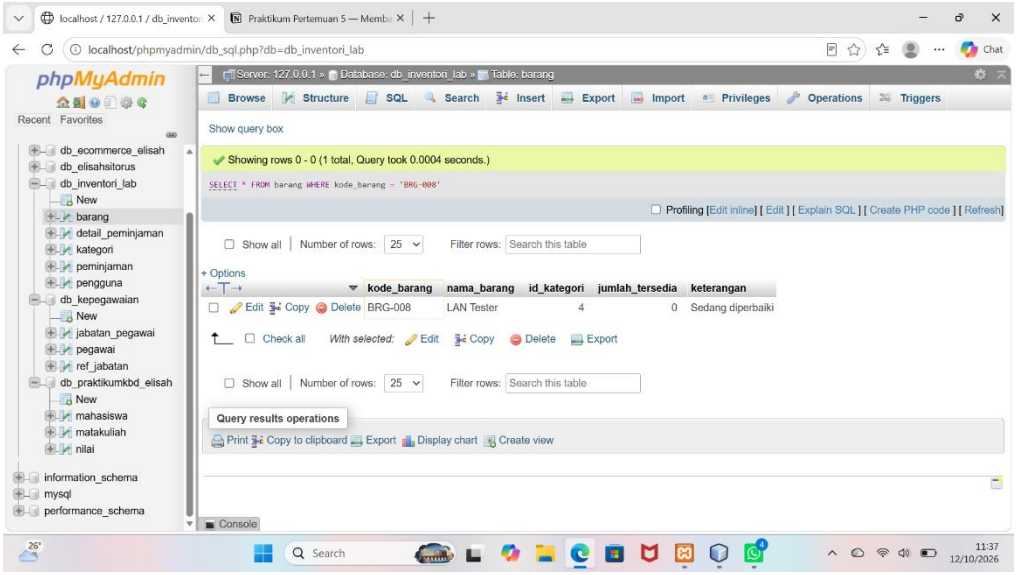
Hasil: Hasilnya 3 baris: transaksi PJM-0002 atas nama Siti Rahma (tanggal 2026-10-08) meminjam Keyboard USB 2 unit dan Mouse USB 2 unit, dilayani Petugas Lab Satu; transaksi PJM-0003 atas nama Budi Santoso (2026-10-09) meminjam Switch 8 Port 1 unit, dilayani Administrator Lab.

9. Memeriksa Data Sebelum UPDATE

Sebelum mengubah data, baris yang akan diubah dicek dulu agar tidak salah sasaran.

Perintah SQL:

```
SELECT * FROM barang
WHERE kode_barang = 'BRG-008';
```



Hasil: Kondisi

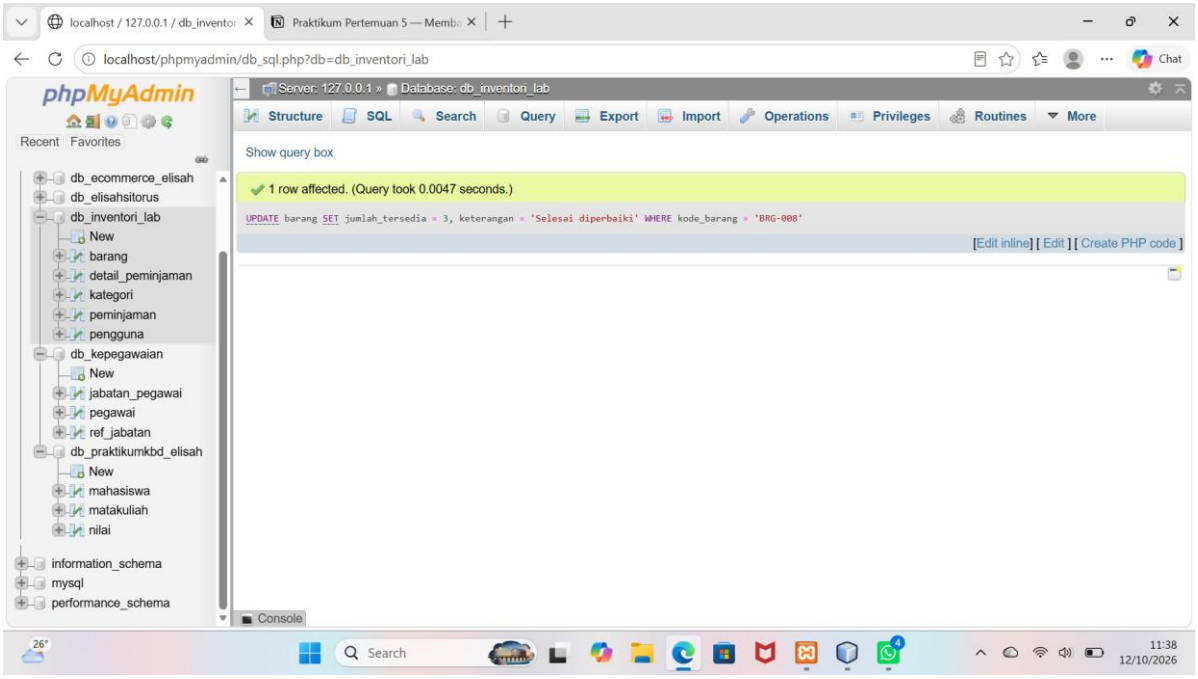
awal ini menjadi pembanding untuk melihat perubahan setelah perintah UPDATE.

10. Mengubah Data dengan UPDATE

Perintah UPDATE dipakai untuk memperbarui data yang sudah ada. Di sini LAN Tester yang selesai diperbaiki dicatat kembali sebagai tersedia.

Perintah SQL:

```
UPDATE barang
SET jumlah_tersedia = 3,
    keterangan = 'Selesai diperbaiki'
WHERE kode_barang = 'BRG-008';
```



Hasil: Data BRG-008 berhasil diperbarui, dan hasilnya terlihat pada tampilan data barang di bagian 2.

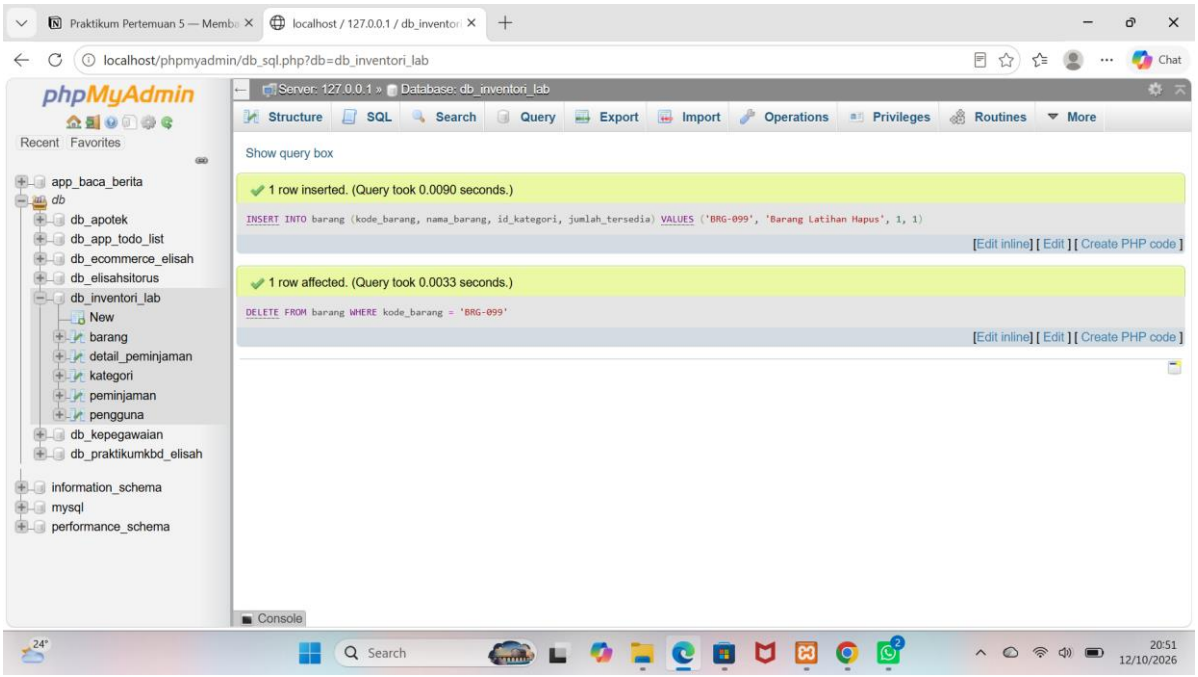
11. Menambah Data dengan INSERT dan Menghapus dengan DELETE

Dua perintah dijalankan berurutan: menambah satu barang latihan, lalu menghapusnya.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-099', 'Barang Latihan Hapus', 1, 1);

DELETE FROM barang
WHERE kode_barang = 'BRG-099';
```



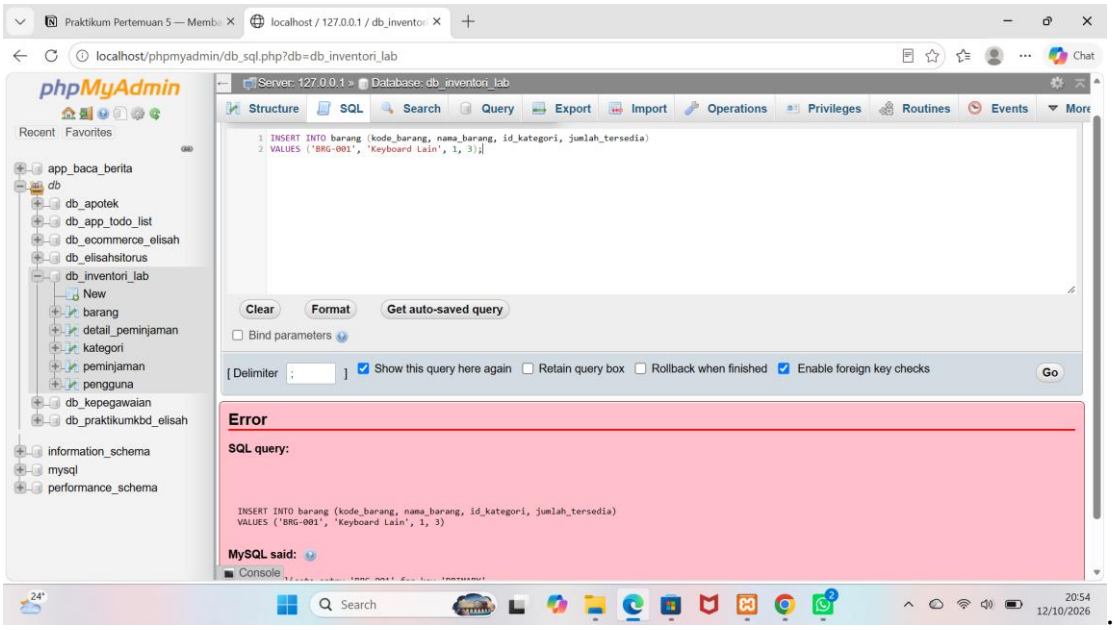
Hasil: Data latihan berhasil ditambah lalu dihapus, dan isi tabel barang kembali seperti semula.

12. Uji Error 1: Primary Key Duplikat

Pengujian ini sengaja memasukkan kode_barang yang sudah ada untuk membuktikan primary key bekerja.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-001', 'Keyboard Lain', 1, 3);
```



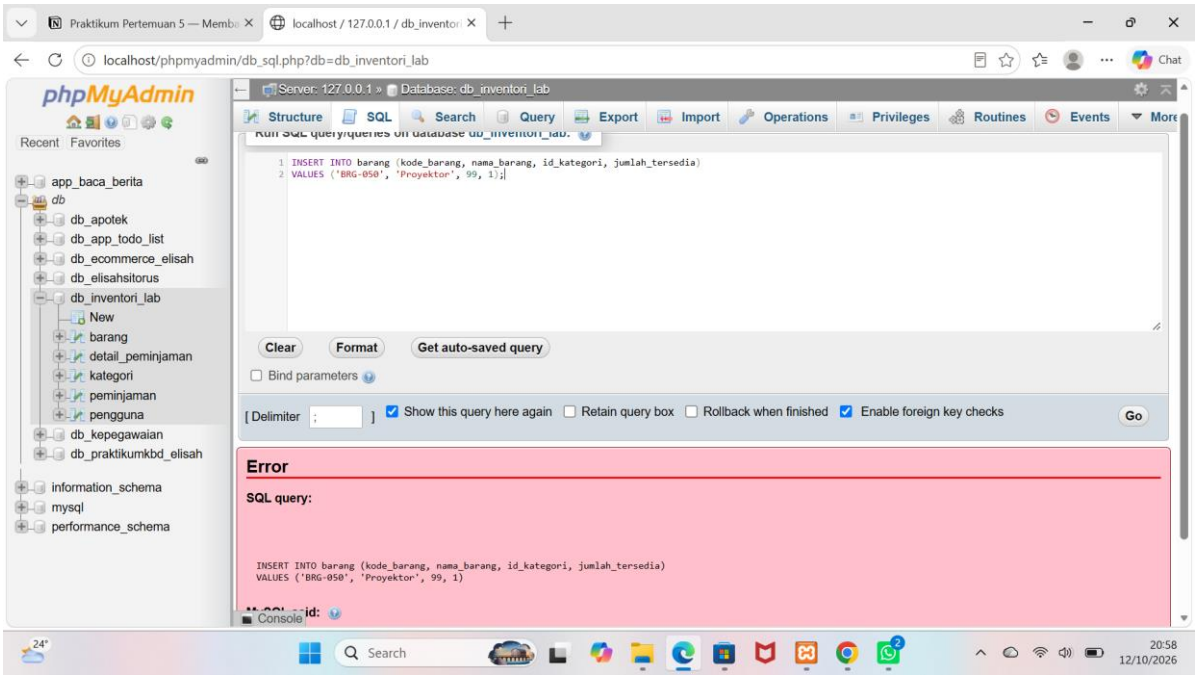
Hasil: Query gagal dan data tidak berubah. Ini membuktikan primary key mencegah adanya dua barang dengan kode yang sama.

13. Uji Error 2: Pelanggaran Foreign Key (Kategori Tidak Ada)

Pengujian ini memasukkan barang dengan id_kategori 99, padahal kategori tersebut tidak ada di tabel kategori.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-050', 'Proyektor', 99, 1);
```



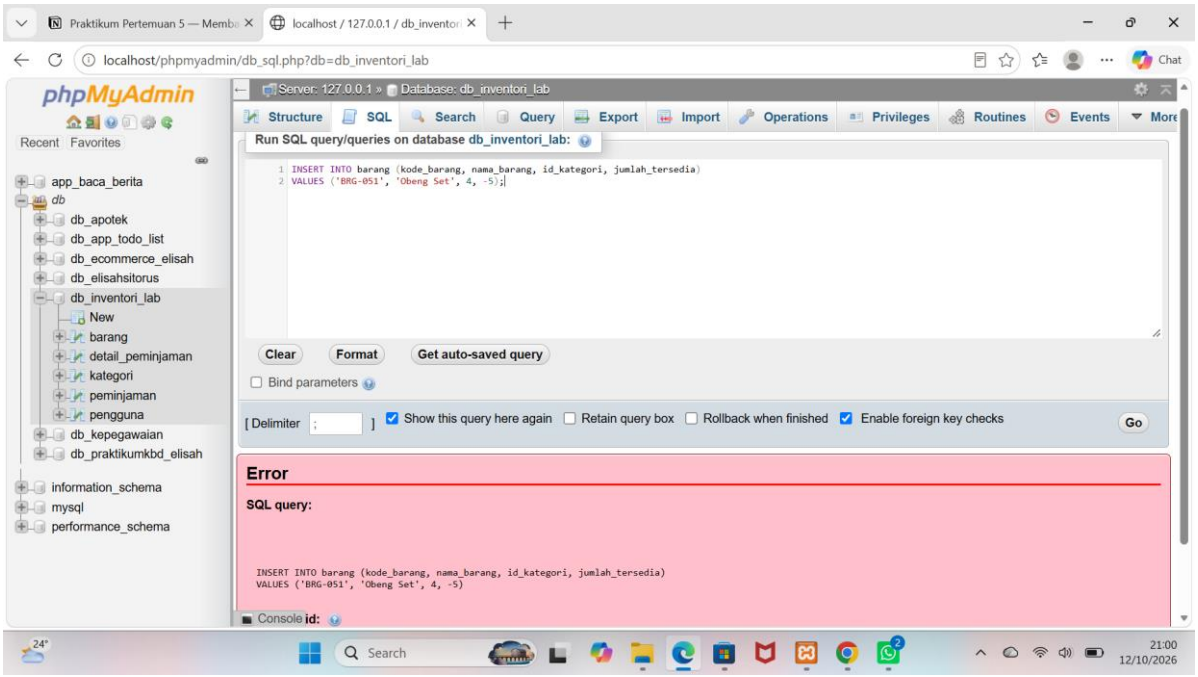
Hasil: Query gagal. Foreign key menjaga integritas referensial sehingga setiap barang pasti terhubung ke kategori yang valid.

14. Uji Error 3: Pelanggaran CHECK Constraint (Stok Negatif)

Pengujian ini memasukkan stok bernilai negatif (-5), yang tidak masuk akal untuk data persediaan.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-051', 'Obeng Set', 4, -5);
```



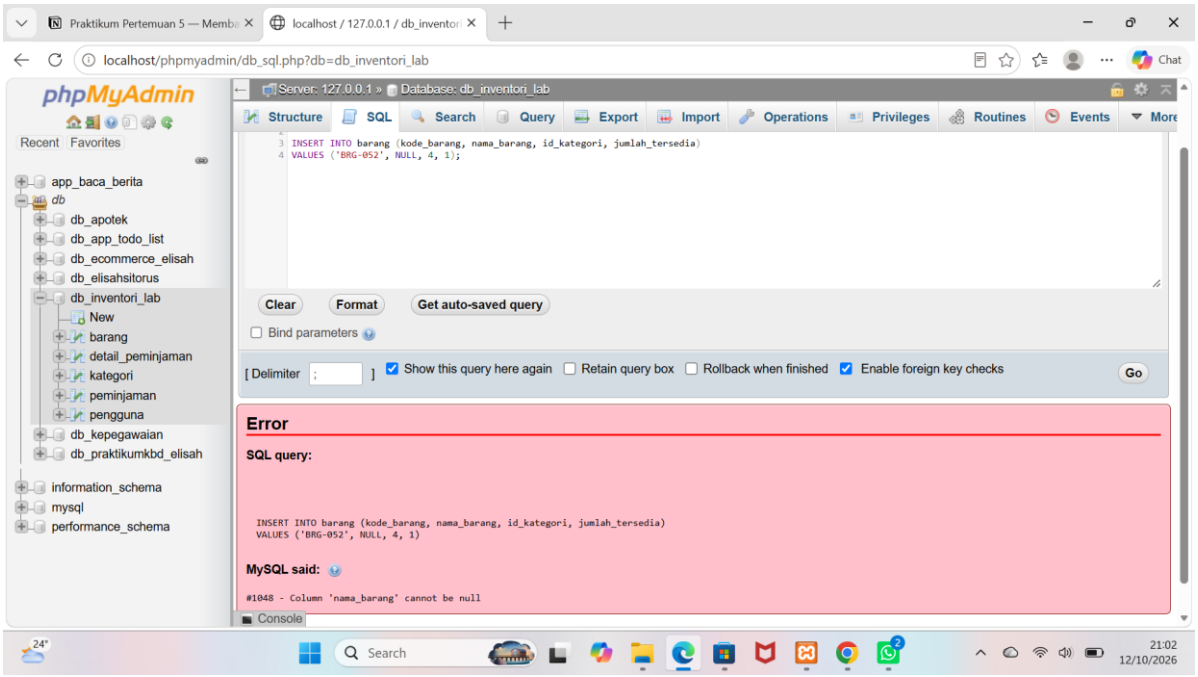
Hasil: Query gagal. CHECK constraint memastikan nilai stok selalu valid.

15. Uji Error 4: Kolom NOT NULL Diisi NULL

Pengujian ini mengisi nama_barang dengan NULL (kosong).

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-052', NULL, 4, 1);
```



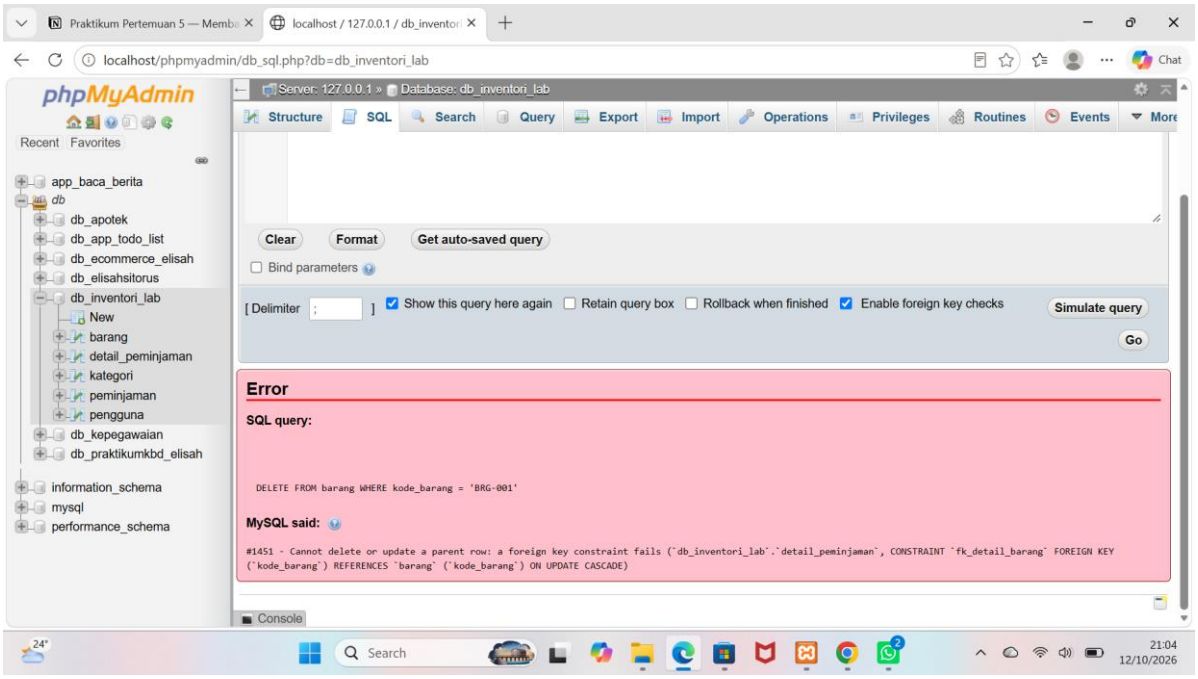
Hasil: Query gagal. Aturan NOT NULL mencegah data barang tersimpan tanpa nama.

16. Uji Error 5: Menghapus Barang yang Masih Dipakai (Foreign Key)

Pengujian ini mencoba menghapus Keyboard USB (BRG-001) yang sudah tercatat pada transaksi peminjaman.

Perintah SQL:

```
DELETE FROM barang
WHERE kode_barang = 'BRG-001';
```



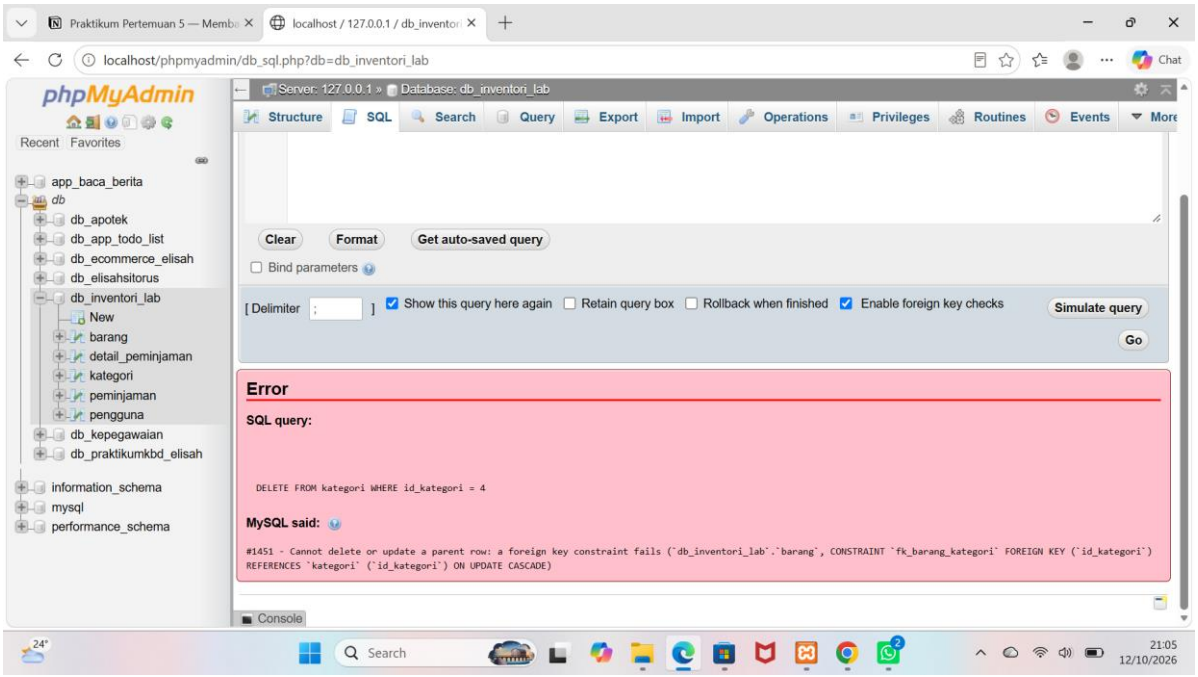
Hasil: Query gagal. Riwayat peminjaman tidak kehilangan data barang yang direferensikannya.

17. Uji Error 6: Menghapus Kategori yang Masih Dipakai Barang

Pengujian ini mencoba menghapus kategori dengan id 4 (Alat Ukur) yang masih dipakai oleh tabel barang.

Perintah SQL:

```
DELETE FROM kategori
WHERE id_kategori = 4;
```



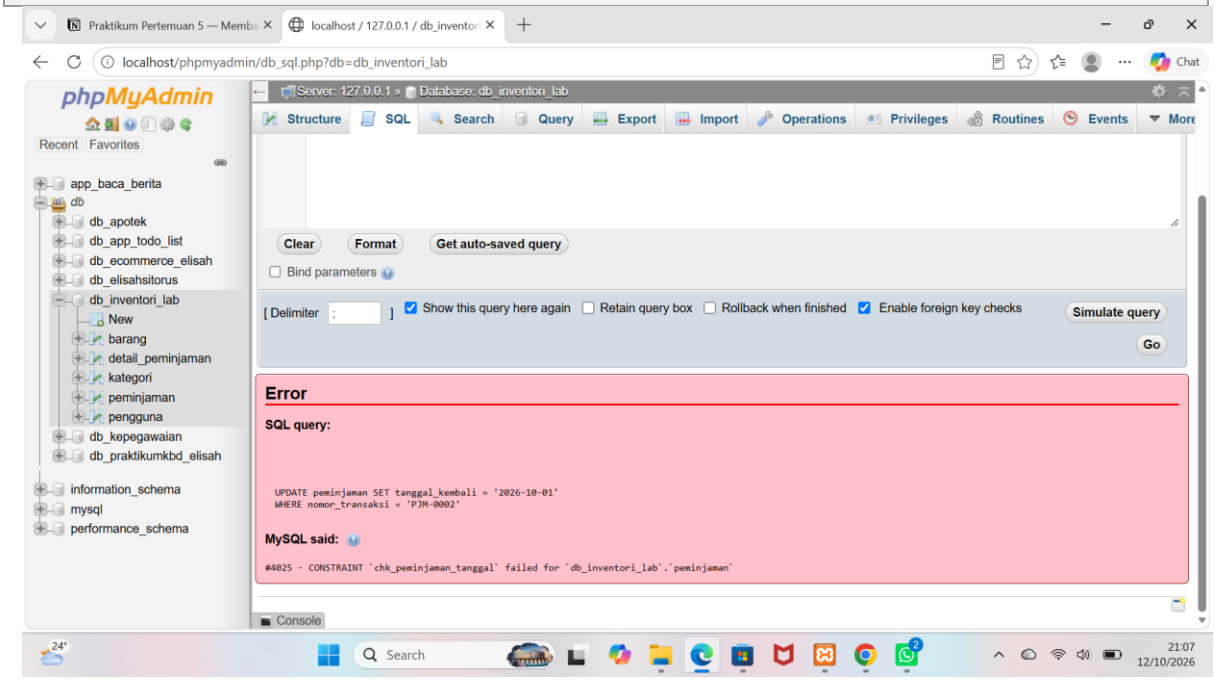
Hasil: Query gagal. Agar kategori dapat dihapus, barang yang memakainya harus dihapus atau dipindah kategori terlebih dahulu.

18. Uji Error 7: Pelanggaran CHECK pada Tanggal Peminjaman

Pengujian ini mengisi tanggal kembali (2026-10-01) yang lebih awal daripada tanggal pinjam transaksi PJM-0002 (2026-10-08).

Perintah SQL:

```
UPDATE peminjaman
SET tanggal_kembali = '2026-10-01'
WHERE nomor_transaksi = 'PJM-0002';
```



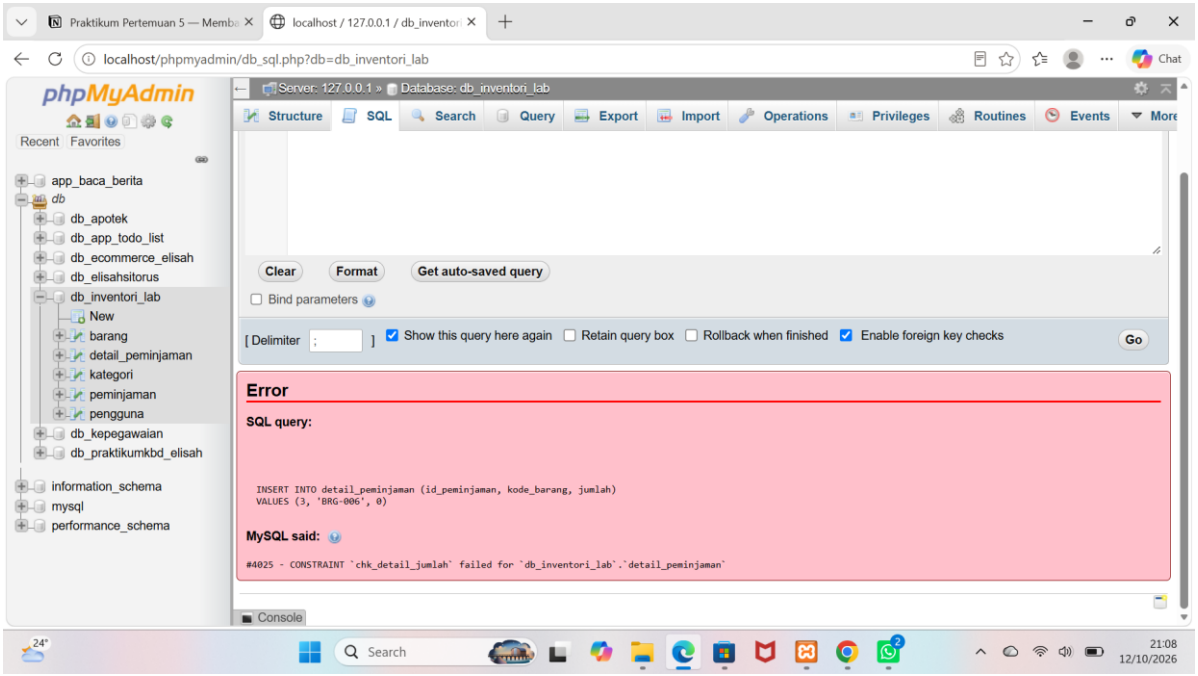
Hasil: Query gagal. Data tanggal peminjaman tetap logis.

19. Uji Error 8: Pelanggaran CHECK pada Jumlah Detail Peminjaman

Pengujian ini mencatat peminjaman barang dengan jumlah 0.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (3, 'BRG-006', 0);
```



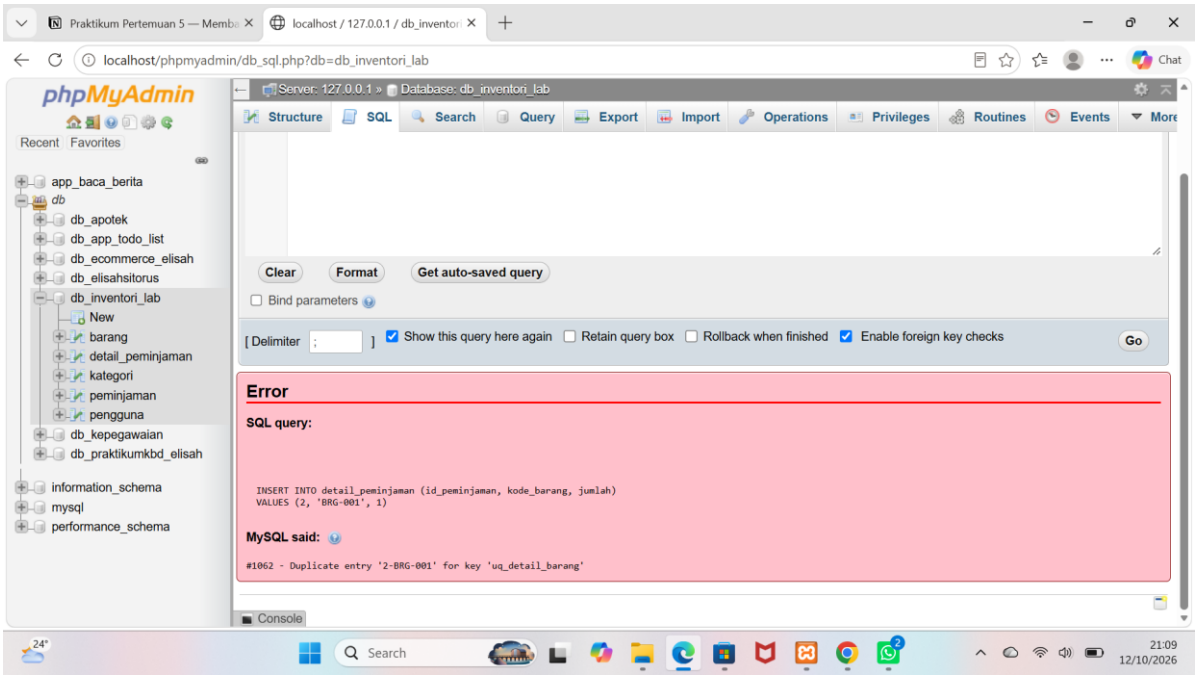
Hasil: Query gagal. Tidak mungkin ada peminjaman yang jumlah barangnya nol.

20. Uji Error 9: Unique Key Ganda pada Detail Peminjaman

Pengujian ini mencoba mencatat barang BRG-001 (Keyboard USB) untuk transaksi dengan id_peminjaman 2 (PJM-0002), padahal barang tersebut sudah tercatat di transaksi itu.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (2, 'BRG-001', 1);
```



Hasil: Query gagal. Unique key mencegah baris detail ganda untuk barang yang sama dalam satu transaksi.

21. Menambah Data Peminjaman Latihan beserta Detailnya

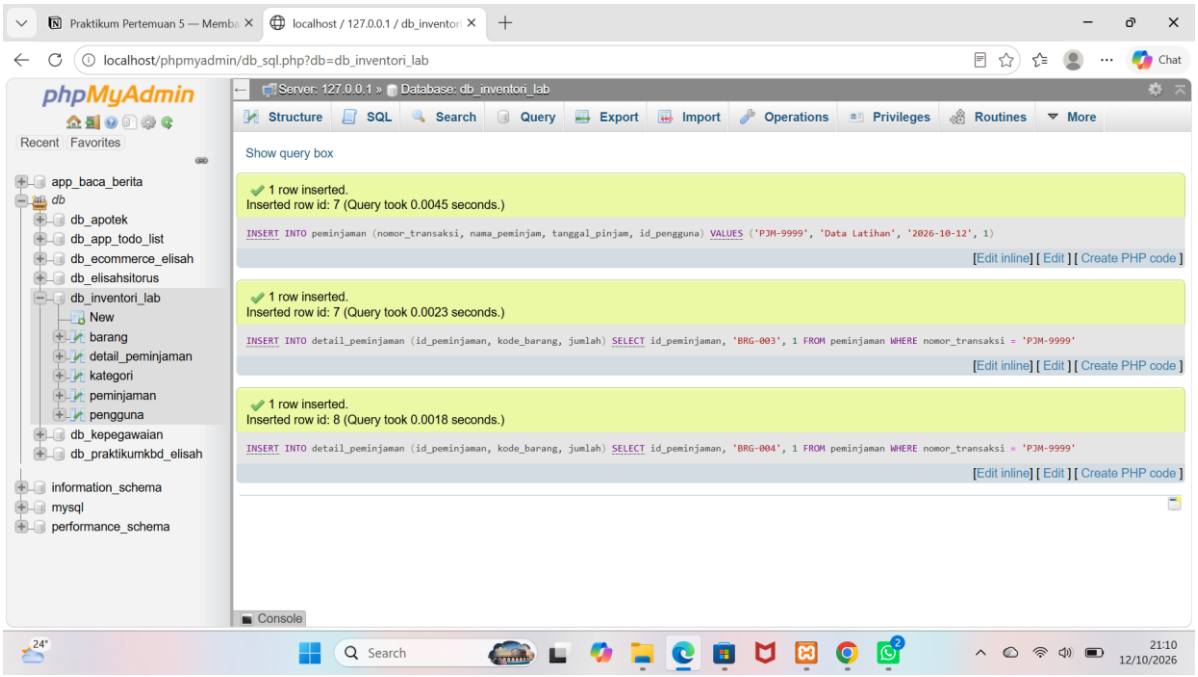
Tiga perintah INSERT dijalankan berurutan untuk membuat satu transaksi latihan (PJM-9999) yang memiliki dua barang pada tabel detail.

Perintah SQL:

```
INSERT INTO peminjaman (nomor_transaksi, nama_peminjam, tanggal_pinjam,
id_pengguna)
VALUES ('PJM-9999', 'Data Latihan', '2026-10-12', 1);

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-003', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-004', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';
```



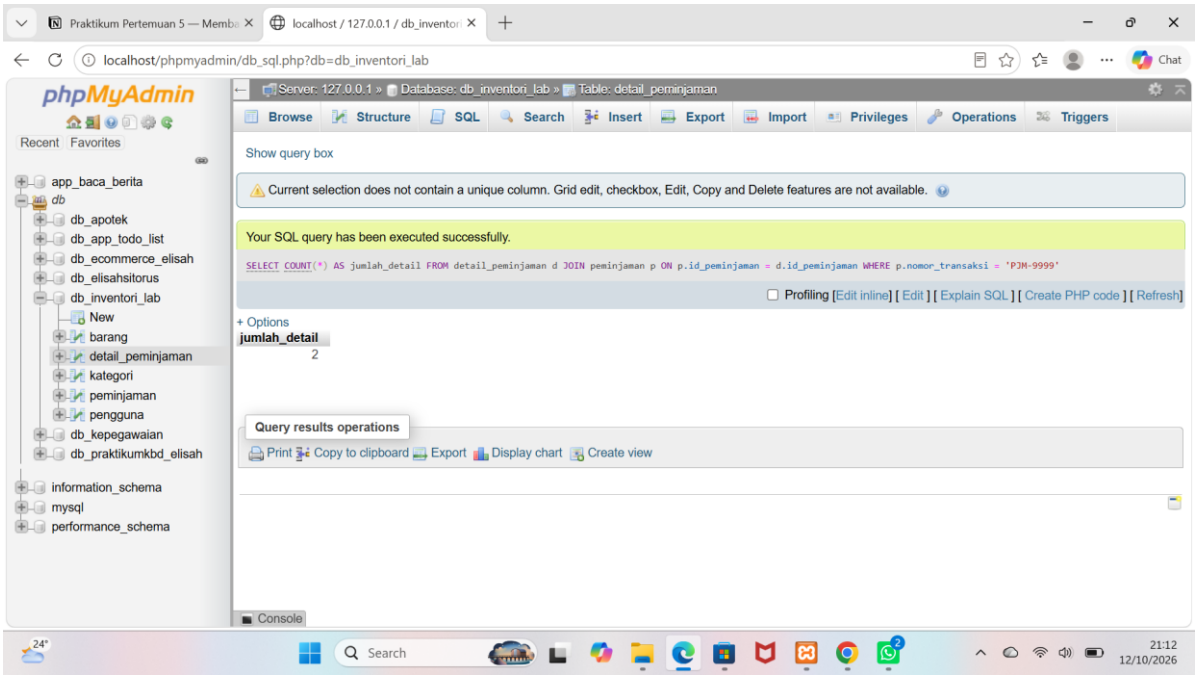
Hasil: Ketiga perintah berhasil (1 row inserted pada masing-masing). Data ini dipakai untuk menguji penghapusan berantai pada langkah berikutnya.

22. Menghitung Jumlah Detail Transaksi PJM-9999

Query ini memastikan transaksi PJM-9999 benar-benar memiliki baris detail sebelum transaksinya dihapus.

Perintah SQL:

```
SELECT COUNT(*) AS jumlah_detail
FROM detail_peminjaman d
JOIN peminjaman p ON p.id_peminjaman = d.id_peminjaman
WHERE p.nomor_transaksi = 'PJM-9999';
```



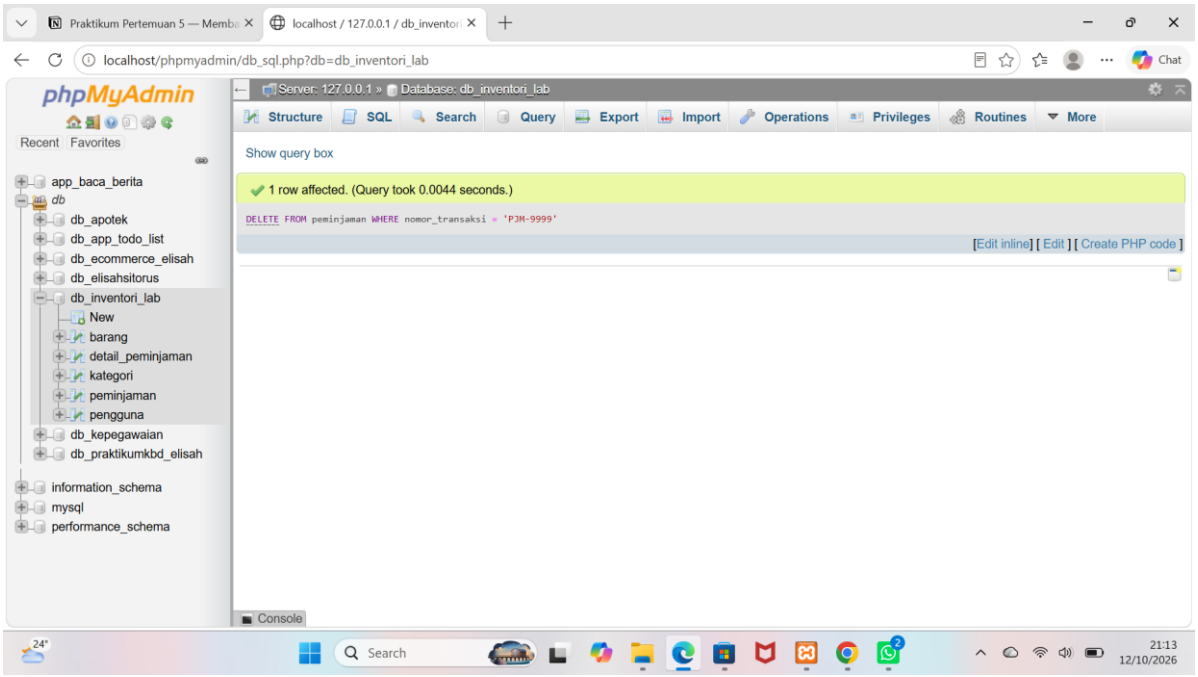
Hasil: Terbukti ada 2 baris detail untuk PJM-9999 sebagai kondisi awal sebelum DELETE.

23. Menghapus Transaksi Peminjaman (DELETE pada Tabel Induk)

Transaksi latihan PJM-9999 dihapus dari tabel peminjaman, padahal transaksi ini masih memiliki 2 baris detail di tabel anak.

Perintah SQL:

```
DELETE FROM peminjaman
WHERE nomor_transaksi = 'PJM-9999';
```



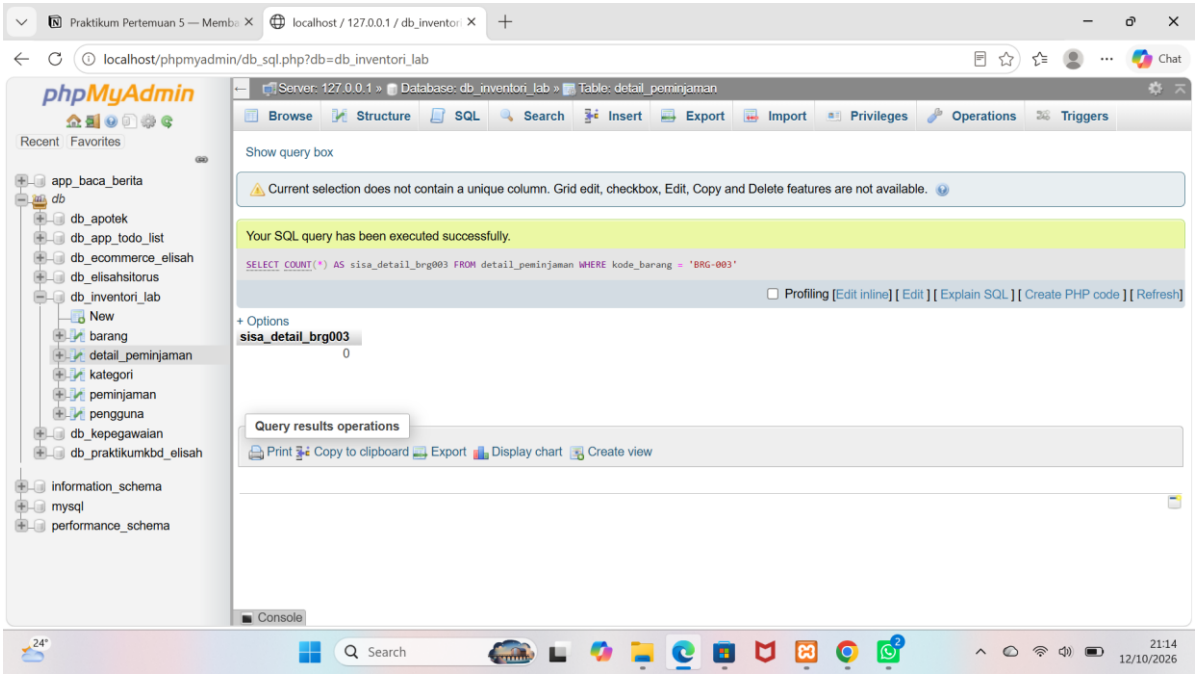
Hasil: Transaksi PJM-9999 berhasil dihapus. Pembuktian bahwa baris detailnya ikut terhapus ada pada langkah berikutnya.

24. Pembuktian Penghapusan Berantai (CASCADE) pada Detail

Query ini mengecek apakah baris detail milik PJM-9999 masih tersisa setelah transaksi induknya dihapus. BRG-003 dipilih karena hanya dipakai pada transaksi PJM-9999.

Perintah SQL:

```
SELECT COUNT(*) AS sisa_detail_brg003
FROM detail_peminjaman
WHERE kode_barang = 'BRG-003';
```



Hasil: Penghapusan berantai terbukti bekerja: menghapus data induk pada tabel peminjaman otomatis membersihkan data anak pada tabel detail_peminjaman.